Take-Home Assignment: RevOps Engineer

Stargo · RevOps Engineer (AI-native Marketing Brain) · Candidate brief v2

Thanks for taking the time to work on this. We designed it to be close to real work rather than a puzzle: a slightly ambiguous spec, messy GTM data, real tool choices, and a chance to show how you build in 2026, including how you use AI coding tools. Everything in the data pack is synthetic.

How you prioritize is part of the assignment. We score depth on the core over breadth. If you choose to cut something, list what you cut and why in the README.

Scenario

Stargo's GTM team runs on a weekly rhythm. Replies, forms and LinkedIn messages arrive across channels. Someone has to decide, fast and consistently, which of them are real qualified leads, who owns each one, and what the week's numbers actually are. At the same time, buyers increasingly ask ChatGPT, Perplexity and Gemini which vendors to consider, and we need to know where Stargo is missing from those answers.

You will build the first version of one small app that turns raw GTM signals into qualified, routed leads, an honest weekly KPI pack, and a ranked list of AI-visibility gaps.

What to build

1. Ingest and hygiene

- Load `crm_export.csv` and `gtm_events.json` on first run.
- Let a user paste a new inbound message: sender name, email or LinkedIn URL (either may be blank), channel and text, plus an optional received time (defaults to now) and an optional source (blank means not recorded). It must flow through the same pipeline as the seed data.
- Normalize and match identities and domains as defined in `ql_standard.md`. Reject invalid records with a clear message, without crashing.

2. Qualification pipeline: two agents, chained

Agent A: Signal Extraction. Input: one inbound message plus the matched CRM record, if any. Output: structured JSON with language, intent (demo, pricing, pilot, not interested, unsubscribe, auto-reply, partnership, investor, job seeker, other), stated buyer role, stated data problem, stated company facts, and short evidence quotes from the message.

Agent B: Qualification and Routing. Input: Agent A's validated output plus CRM facts. Never the raw message alone. Output: a 0 to 100 priority score with a per-factor breakdown; a status from section 5 of `ql_standard.md`, checked against all five QL conditions; the owner from the routing table; a one-line rationale tied to evidence; and, where appropriate, a short first-reply draft that a human must approve. Nothing is ever sent.

Validate each agent's output against a schema (Pydantic, Zod or similar) before it moves on. You decide which parts are deterministic code and which are LLM calls. Explain the split. The QL gate and exclusions must be auditable.

Handle the unhappy path: model failure, timeout, invalid JSON, non-English input, and instructions hidden inside a message. The app must keep working, message content must never change the rules, and a failed analysis shows as `needs review`, never as QL or disqualified.

3. Weekly KPI pack

Compute the pack for **Sun 4 Oct to Thu 8 Oct 2026 (Asia/Jerusalem)** per section 6 of `ql_standard.md`: QLs, meetings booked, meetings held and opportunities versus target, QLs by first-touch source, and expansion on its own line. Each number must be traceable to the records behind it. Unknown is not zero.

The pack is recomputed from stored decisions, so an Accept or Reject made in the UI changes it only if its `decided_at` falls inside the week.

4. AI visibility snapshot: GEO / LLM-SEO

From `ai_answers.json`:

- Per prompt and engine: is Stargo mentioned, is it cited from an owned domain, which competitors appear. Watch for look-alike brands and wrong claims about Stargo.
- A ranked gap list using a scoring formula you define and justify.
- For the top gap, an LLM-generated content brief that follows the GEO content contract in `brand_context.md`. Every claim must cite a BC or PP id. Unsupported claims must be blocked or flagged.

5. Storage

Persist leads, agent outputs, decisions and the KPI pack in SQLite or a JSON file. Everything must survive a restart.

6. UI: React + TypeScript

- **Lead queue:** name, company, channel, received time (Israel time), priority score badge, status, owner.
- **Lead detail:** original message, Agent A extraction, score breakdown, QL checklist with evidence, rationale, reply draft. **Accept** / **Reject** records a sales decision with a `decided_at` that defaults to now and can be edited, so a past week can be replayed.
- **KPI pack view:** actual vs target, by source, with drill-down to records.
- **AI visibility view:** gap table and the generated brief.
- Readable at a 390 px wide viewport. No mobile polish needed.

Tests (written alongside each part)

Cover at least: the QL gate, identity matching, the reporting window across time zones, KPI counting, and one LLM failure path (for example invalid JSON ending in `needs review`). Fake the model in tests so they run offline.

Bonus (optional, not needed for a top score): a dry-run HubSpot sync that outputs the exact CRM v3 API payloads you would send (contact updates, lead status, owner) without calling any real portal; or a small labelled eval set that measures your qualifier against the QL standard.

Tech constraints

- **Backend:** Python (FastAPI) or Node.js/TypeScript (Express or similar).
- **Frontend:** React + TypeScript. Any styling approach; justify it briefly in the README.
- **LLM:** free tier or local, no paid key required (for example Groq free tier, Gemini free tier, or Ollama). Ship a `.env.example`. The app must run end to end without any key, using a clearly labelled fallback (rules only or recorded fixtures).
- No auth, no real CRM, email or social accounts. Do not send anything anywhere.

How we will review it

No surprises. We will:

1. Follow your README verbatim with no LLM key, then with a free-tier key if we have one.
2. Check `KPI_PACK.md` and the KPI view against the data, and drill into two numbers.
3. Paste a few new messages you have not seen through the UI: some non-English, some edge cases, one with hidden instructions. Build for the rules, not for the seed records.
4. Simulate LLM failure: invalid key, unreachable host, garbage or slow responses.
5. Accept and reject leads with past and current `decided_at` times, restart, and check the KPI pack and persistence.
6. Check logs for message bodies, emails and prompts, and check that network calls go only to your chosen LLM endpoint.
7. Meet you for a live session without AI assistants: walk through your code and talk through a small change.

What we're evaluating

Dimension	Weight
Requirements coverage (parts 1 to 3 end to end; parts 4 to 6 working or cut with a reason)	15
Architecture and design (ingestion, agents, rules, storage, API, UI clearly separated)	10
Code quality (types, schema validation, small modules, linting)	10
Correctness and data integrity (identity, counting, time zones, edge records, unseen messages)	15
Testing (the five areas listed above)	10

Dimension	Weight
Security and privacy (prompt injection, PII in logs, human-in-the-loop outbound, secrets)	10
DevEx and operability (setup, fallback mode, README)	5
AI-assisted engineering maturity (`PROCESS.md`)	10
GTM product thinking and communication (operator trade-offs, CRO note)	15
Total	**100**

How you used AI tooling is a real part of the grade. Your `PROCESS.md` (or an exported Cursor or agent plan) should show your planning, real prompts, where you let an agent run, where you stepped in and corrected it, and how you checked its output. Write it yourself, in the first person.

We also check, outside the score: the app runs per your README, core requirements are met, no secrets are committed, nothing is sent externally, the work is your own with AI use disclosed, and your time report is honest.

Deliverables

- A git repository (GitHub link, or a zip that includes the `.git` folder so we can see history).
- `README.md` : setup and run, architecture, the deterministic vs LLM split, scoring formulas, key assumptions and trade-offs, what you cut, and what you would do with more time.
- `PROCESS.md` : your AI workflow, as described above.
- `KPI_PACK.md` : the KPI pack your app produces for the week under review, as exported output, ending with a **note to the CRO** of 150 words or fewer: the week against target in plain words, the top AI-visibility gap and why it matters to pipeline, and the one action you would take next week.
- Optional: a short screen recording explaining your design, not only touring features.

Submission

- Reply to the email that delivered this brief, with your repository link, the total hours you spent, and the LLM you used.
- Submit within 5 calendar days of receiving the brief, unless we agreed otherwise.
- Do not commit `.env` files or keys. Keep the repository private and invite the reviewer if you prefer.
- Do not push commits after you submit. If you must, tell us what changed and why.

Prioritization

We do not expect production polish. We expect good judgment about what to prioritize and how you manage your own time. A clearly reasoned cut in parts 4 to 6 costs less than a shallow version of everything. Tell us honestly how long you spent; it helps us calibrate the task, and any question about it is a conversation, not a penalty.

Data pack

`crm_export.csv` , `gtm_events.json` , `ai_answers.json` , `ql_standard.md` , `brand_context.md` , `DATA_README.md` . Some records are deliberately tricky. Add your own test cases if useful.

Questions during the task are welcome. Reply to the same email thread at any time.